1. Dominio y objetivo

Arquitectura orientada a servicios para gestionar materiales de un club deportivo. API Gateway, microservicio REST y base de datos persistente.

Autor	Tecnología asignada	Fecha
Andrés Fernando Zuxo Condori	C# / ASP.NET Core 8	19 de octubre de 2026

Problema de negocio

Un club deportivo necesita identificar sus materiales, sus categorías, precios y existencias. Los registros dispersos dificultan conocer cantidades y evitar códigos duplicados. El sistema centraliza este catálogo y aplica reglas de integridad antes de guardar la información.

Objetivo y alcance

Diseñar, construir y desplegar una solución con un único punto de entrada HTTP. El Gateway aplica políticas comunes; el backend resuelve el negocio y PostgreSQL conserva los datos. Se implementa CRUD completo para categorías y productos, documentado en Swagger y ejecutable con Docker Compose.

El alcance se concentra en el catálogo. No incluye ventas, pagos, facturación ni un libro de movimientos de almacén. Las existencias se modifican mediante la actualización del producto. Los precios se expresan en bolivianos.

Modelo y reglas principales

Entidad	Datos y reglas
Categoría	ID y nombre único, obligatorio, de 2 a 80 caracteres.
Producto	ID, nombre, SKU único, precio, stock y categoría obligatoria.
Relación	Una categoría tiene muchos productos; se restringe su eliminación cuando está asociada.
Integridad	SKU normalizado a mayúsculas; precio y stock no negativos; índices únicos y clave foránea.

Resultado verificado: 24 comprobaciones de integración satisfactorias. La documentación distingue esas pruebas ejecutadas de las decisiones de diseño y de las limitaciones del alcance.

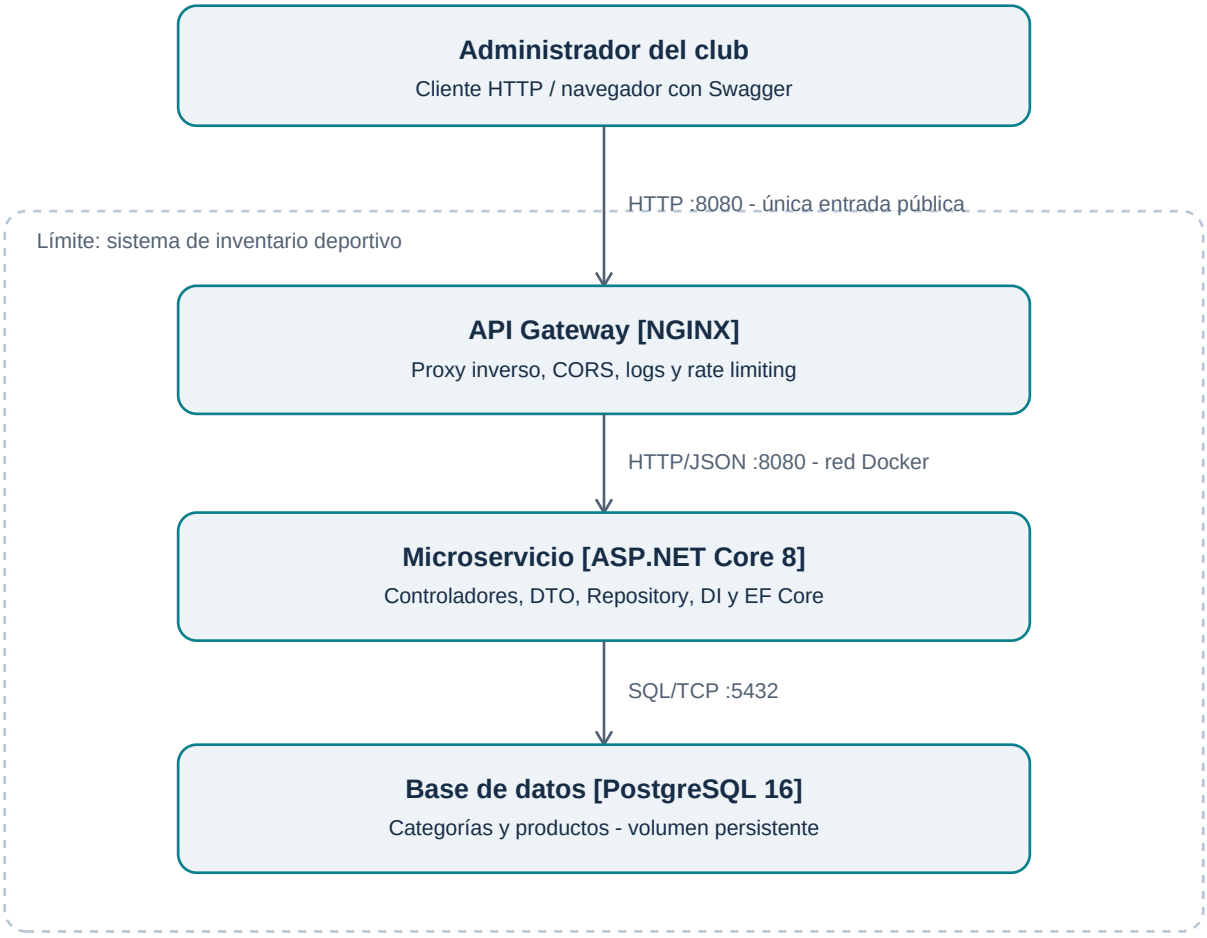
2. Arquitectura C4

Nivel 1 - Contexto del sistema



El administrador es el usuario del sistema. No hay sistemas externos integrados en el caso de uso seleccionado.

Nivel 2 - Contenedores



Los tres contenedores comparten una red bridge. Solo NGINX publica puerto al host. Swagger forma parte del backend y se accede a través del Gateway; no representa un cuarto contenedor.

3. Implementación y Docker

Funciones transversales en el Gateway

Función	Implementación
CORS	Permite localhost:8080, 127.0.0.1:8080 y localhost:5173. Preflight OPTIONS responde 204.
Logs centralizados	stdout en JSON: fecha, método, URI, estado, duración, upstream y requestId.
Rate limiting	10 solicitudes por segundo e IP, ráfaga de 30; exceso con HTTP 429.
Trazabilidad	NGINX asigna X-Request-ID, lo propaga al backend y lo devuelve al cliente.

Organización del backend

Los controladores exponen REST y coordinan las reglas del dominio. Los DTO validan las entradas y separan el contrato público de las entidades. IRepository<T> abstrae las operaciones comunes e IProductRepository incorpora la lectura de productos con su categoría. La inyección de dependencias registra los repositorios y DbContext por solicitud.

EF Core utiliza el proveedor PostgreSQL. El modelo establece índices únicos, una clave foránea con borrado restringido y restricciones de precio y stock. Las validaciones se asocian a los parámetros de los DTO record. Los conflictos de integridad devuelven 409.

Contenerización y arranque

Servicio	Construcción, salud y dependencias
db	postgres:16-alpine; pg_isready; volumen inventory_data.
backend	Dockerfile multietapa: SDK 8 para compilar y runtime ASP.NET 8; usuario app; curl /health comprueba conexión a BD. Espera db saludable.
gateway	nginx:1.28-alpine; wget /health a través del proxy. Espera backend saludable. Publica 8080:8080.

El healthcheck se complementa con depends_on condicionado a service_healthy. Los servicios usan restart: unless-stopped. El volumen nombrado conserva los datos cuando se ejecuta docker compose down sin eliminar volúmenes.

Límite de evolución: esta primera versión crea el esquema mediante EnsureCreatedAsync. No incorpora migraciones de esquema ni autenticación de usuarios; el Gateway sí implementa las tres funciones transversales descritas.

4. Endpoints y contratos JSON

Base: **http://localhost:8080**. Swagger: **/swagger**. Especificación importable en Postman: **/swagger/v1/swagger.json**.

Método	URL	Éxito / respuesta
GET	/api/categories	200 / colección JSON
GET	/api/categories/{id}	200 / categoría JSON
POST	/api/categories	201 / JSON y Location
PUT	/api/categories/{id}	204 / sin contenido
DELETE	/api/categories/{id}	204 / sin contenido
GET	/api/products	200 / colección JSON
GET	/api/products/{id}	200 / producto JSON
POST	/api/products	201 / JSON y Location
PUT	/api/products/{id}	204 / sin contenido
DELETE	/api/products/{id}	204 / sin contenido

Ejemplos de solicitudes

Categoría:

```
{"name":"Equipamiento de entrenamiento"}
```

Producto (utilizar una categoría existente):

```
{"name":"Balon de futbol", "sku":"BAL-001",
"price":75.50, "stock":12, "categoryId":4}
```

Respuesta de producto:

```
{"id":1, "name":"Balon de futbol", "sku":"BAL-001",
"price":75.50, "stock":12, "categoryId":4,
"categoryName":"Equipamiento de entrenamiento"}
```

Semántica HTTP y validaciones

400: datos inválidos o categoría inexistente. 404: recurso inexistente. 409: SKU o nombre duplicado, o categoría con productos asociados. 429: exceso de solicitudes. POST devuelve Location; PUT reemplaza los campos editables completos. Los IDs de los ejemplos son ilustrativos, salvo la categoría 4 observada en la captura.

Los listados están limitados a 1000 registros ordenados por ID. Precio: entre 0 y 9999999.99, almacenado con dos decimales. Stock: entero de 0 a 1000000.

5. Pruebas de integración

Ejecución real del 19 de octubre de 2026 en Windows PowerShell. El script probar.ps1 ejecuta solicitudes a través del Gateway y conserva resultados, logs y estado Docker.

Comprobación	Esperado	Observado
Salud de API y BD	200	200 / OK
Swagger por Gateway	200	200 / OK
Documento OpenAPI	200	200 / OK
Validacion categoria vacia	400	400 / OK
Crear categoria	201	201 / OK
Listar categorias	200	200 / OK
Consultar categoria	200	200 / OK
Editar categoria	204	204 / OK
Crear producto	201	201 / OK
Listar productos	200	200 / OK
Consultar producto	200	200 / OK
Bloquear SKU duplicado	409	409 / OK
Rechazar stock negativo	400	400 / OK
Editar producto	204	204 / OK
Comprobar actualizacion	200	200 / OK
Bloquear categoria con productos	409	409 / OK
Preflight CORS	204	204 / OK
Identificador de peticion	200	200 / OK
Persistencia tras reinicio	200	200 / OK
Eliminar producto	204	204 / OK
Producto eliminado	404	404 / OK
Eliminar categoria	204	204 / OK
Categoria eliminada	404	404 / OK
Rate limiting: ráfaga de 100	Uno o más 429	68 respuestas 429

Resultado: 24/24 comprobaciones satisfactorias. La prueba de persistencia reinició el backend y comprobó que el producto mantenía stock 20. No se destruyó ni recreó el volumen de PostgreSQL.

Trazabilidad de las pruebas: scripts/probar.ps1; evidencias/resultados.json; evidencias/docker-logs.txt. El número de respuestas 429 depende del ritmo y la concurrencia de cada ejecución; 68 es el resultado observado en esta corrida.

6. Evidencia de Swagger

Creación real mediante POST /api/categories, realizada desde Swagger a través del Gateway. La solicitud registra una categoría de equipamiento deportivo.

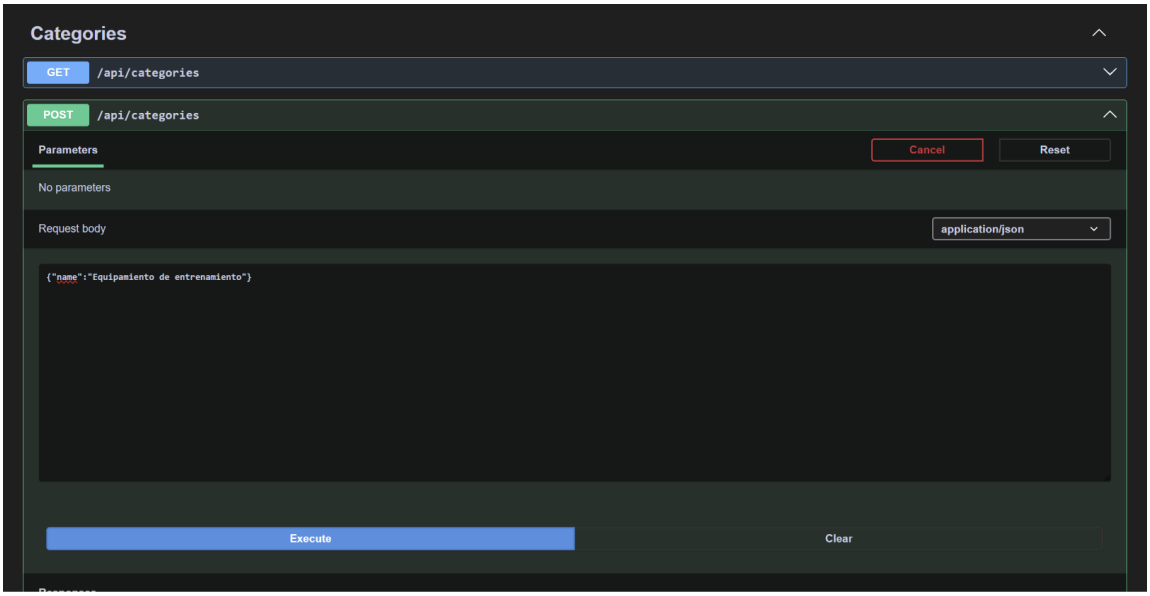


Figura 1. Solicitud JSON introducida en Swagger. Captura original del autor.

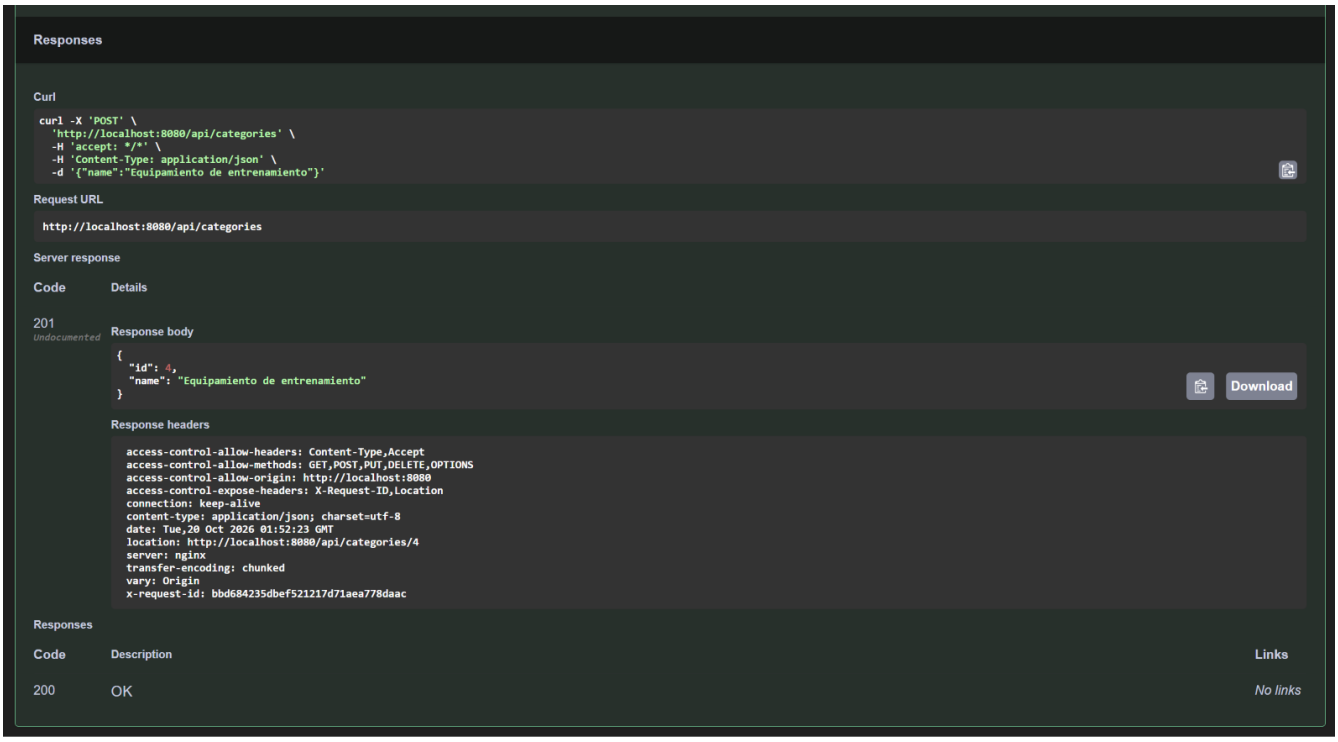


Figura 2. Respuesta **201 Created**, categoría **id: 4**, cabecera Location y X-Request-ID. También se observan las cabeceras CORS y el servidor NGINX.

El código 201 acredita la creación exitosa. Location identifica el recurso creado en /api/categories/4. La batería de pruebas de la página anterior verifica además las operaciones de consulta, modificación y eliminación de las dos entidades.

7. Evidencia Docker y entrega

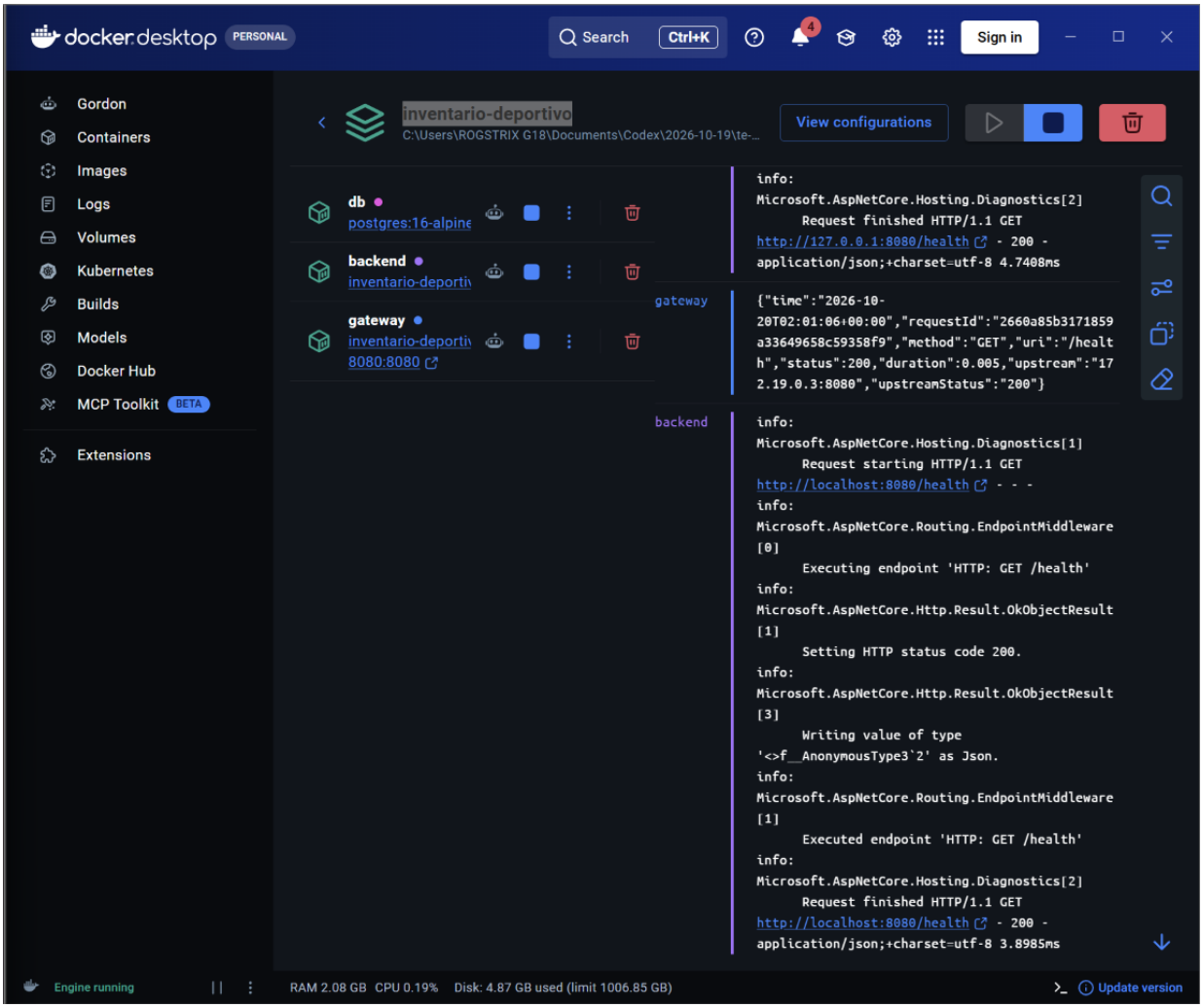


Figura 3. Docker Desktop muestra db, backend y gateway. Los logs incluyen salud HTTP 200 y el registro JSON del Gateway con requestId, duración y upstream. Captura original del autor.

Reproducción y Git Flow

Requisitos: Git, Docker Desktop en modo Linux y Compose. Clonar el repositorio, crear .env a partir de .env.example y ejecutar docker compose up -d --build --wait. En Windows, scripts/iniciar.ps1 automatiza el arranque y las pruebas. El README incluye ejemplos cURL y uso de Swagger/Postman.

Las ramas feature separan backend, Gateway y documentación; develop integra los cambios. La corrección de validación tiene su propio commit y merge. La versión se cierra mediante release/1.0.0, merge a main y etiqueta v1.0.0. El historial registra el desarrollo real.

Conclusión

La solución ejecutada valida la separación entre entrada HTTP, negocio y persistencia. Los dos CRUD, la integridad, CORS, trazabilidad y límite de solicitudes pasaron las pruebas. La entrega se compone del repositorio público y este informe; la publicación se comprueba después de ejecutar git push.

Referencias técnicas

Microsoft Learn: ASP.NET Core 8 y puerto de contenedores.
https://learn.microsoft.com/en-us/dotnet/core/compatibility/containers/8.0/aspnet-port
NGINX: proxy HTTP y límite de solicitudes.
https://nginx.org/en/docs/http/ngx_http_proxy_module.html
https://nginx.org/en/docs/http/ngx_http_limit_req_module.html